

NATURAL LANGUAGE OPINION SEARCH ENGINE

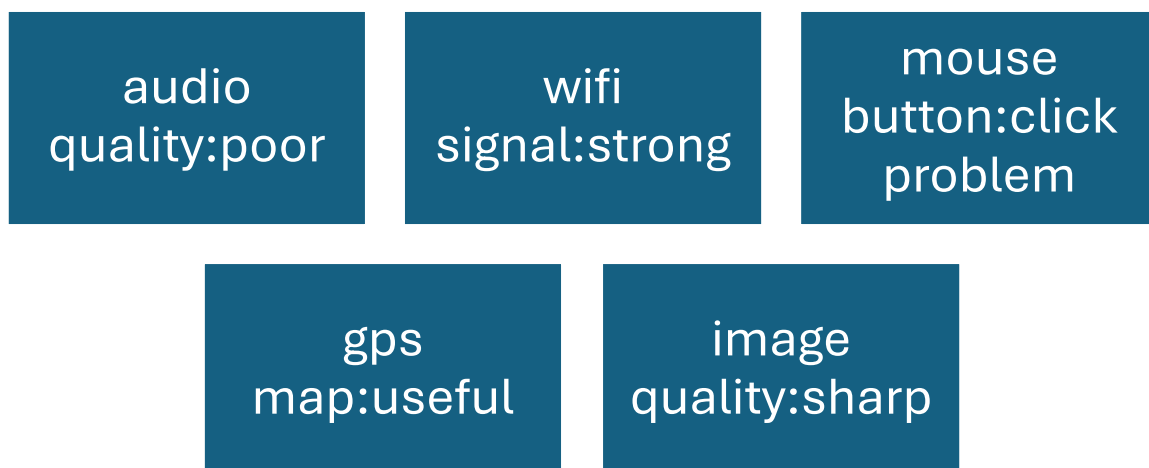


1. Overview and Background

Problem Definition, Corpus, and Research Motivation

When we buy things online we often read what other people think about a product before we make a decision. When we use the search bar it just looks for the words we type in and does not really understand what we are looking for. This project is about making a search engine that can find what people think about certain features of a product. We used a lot of product reviews from Amazon. 210,761 to be exact to make this search engine work. Queries take the structured form *aspect:opinion*, where each component consists of at most two words. The objective is not merely retrieving reviews containing the query terms, but specifically identifying reviews that evaluate the given aspect in the requested sentiment orientation. [5]

The system is evaluated across five queries



Metric	Detail
Total Corpus Reviews	210,761 reviews
Required Queries	5 aspect-opinion queries
Baseline Retrieval Tests	3 tests
Advanced Connotation Model	2 tests

1.1 Why Boolean Retrieval is Necessary but Insufficient

Boolean retrieval gives a clear and easy to check starting point by using patterns that match exact words. Just because words appear together does not mean they are connected in meaning or structure. A review might talk about "quality" in one part and "poor" in another part that's about a different thing like "Great image quality but poor battery life". Also words like "not poor" or ideas that move between sentences can cause mistakes in Boolean systems. The main problem is keeping the amount of results that Boolean search gives while adding sentence-based analysis, from transformer models to catch the right meaning.[1]

1.2 Corpus and Fields Used

The dataset is derived from an Amazon SQL dump containing 210,761 records. The primary fields utilized by the engine are ***review_id*** (used as the unique output identifier) and ***review_text***. The star rating distribution of the underlying corpus is summarized below:

Star Rating	1 Star	2 Stars	3 Stars	4 Stars	5 Stars
Review Count	28,611	15,561	19,224	38,709	108,656

2. Boolean Baseline

To prevent subword false positives such as matching the query term "weak" inside "tweak"; the engine uses strict regex boundary checking combined with ***re.escape()***.

#py code

```
def contains_word(text, word):  
    pattern = rf'\b{re.escape(word)}\b'  
    return bool(re.search(pattern, text, flags=re.IGNORECASE))
```

2.1 Preprocessing and Text Normalization

During text extraction, raw review strings are handled with null-value replacement and string conversion. Standard normalization converts review text to lowercase for case-insensitive matching while preserving original punctuation required for subsequent NLTK sentence tokenization.

2.2 Required Boolean Tests

Let A represent the set of aspect words for a query (e.g., ["audio", "quality"]) and O represent the set of opinion words (e.g., ["poor"]). The baseline engine executes three distinct tests:

1. **Test 1 : Aspect Term Retrieval (A)**: The review must contain at least one aspect term; opinion terms are not required. [5]
2. **Test 2 : Aspect AND Opinion Match ($A \cap O$)**: The review must contain at least one aspect word **AND** at least one opinion word anywhere in the review body.[5]
3. **Test 3 : Aspect OR Opinion Match ($A \cup O$)**: The review must contain at least one term from either the aspect set or the opinion set. [5]

2.3 Strict Output Contract

Output files are written directly to ***../Outputs/Baseline/***. It is ensured that review IDs are saved strictly as one ID per line without any quotation marks, commas, brackets, or line numbers. A total of 15 baseline files are generated across the 5 queries.

```
Outputs/Baseline/audio_quality_test1.txt  
Outputs/Baseline/audio_quality_test2.txt  
Outputs/Baseline/audio_quality_test3.txt  
...  
Outputs/Baseline/image_quality_test3.txt
```

3. Advanced Method

Moving beyond simple term co-occurrence requires evaluating grammatical proximity and local sentiment orientation. The advanced model addresses test 4 using a two-stage approach: sentence tokenization followed by neural sentiment inference.

3.1 NLTK Sentence Tokenization

Because full review texts often span multiple distinct thoughts, analyzing sentiment across an entire document introduces disturbance. The model uses NLTK's *sent_tokenize* to decompose each review into individual sentences.[4]

3.2 DistilBERT Neural Sentiment Classification[3]

To accommodate token sequence limits and prevent memory overflow, input text is truncated to a maximum length of 512 tokens.

Query Name	Aspect Terms	Opinion Terms	Target Sentiment Label
audio_quality	["audio", "quality"]	["poor"]	NEGATIVE
wifi_signal	["wifi", "signal"]	["strong"]	POSITIVE
mouse_button	["mouse", "button"]	["click", "problem"]	NEGATIVE
gps_map	["gps", "map", "maps"]	["useful", "helpful"]	POSITIVE
image_quality	["image", "quality"]	["sharp"]	POSITIVE

3.3 Test 4 Match Logic

A review is included in the test4.txt output if at least one sentence satisfies two conditions simultaneously:

1. The sentence contains at least one aspect word **AND** at least one opinion word.
2. The DistilBERT classifier predicts a sentiment label (POSITIVE or NEGATIVE) that matches the query's target sentiment orientation.

4. System Design and Code Implementation

The system architecture is as follows:

Patel_2447821.zip

```
├── Codes/
│   ├── baseline.py
│   ├── adv.py
│   ├── reviews_segment.sql
│   └── README.md
├── Outputs/
│   ├── Baseline/
│   └── AdvancedModel/
└── Report.pdf
```

4.1 Module Responsibilities^{[3][4]}

Module	Primary Responsibility	Key Libraries Used
Data Parsing	Direct parsing of review records into Pandas DataFrames.	pandas, re
baseline.py	Executes regex-bounded Boolean retrieval for Tests 1, 2, and 3.	pandas, re, os
adv.py	Sentence tokenization and DistilBERT sentiment classification for Test 4.	nltk, transformers, torch

5. Results

The program parsed all 210,761 reviews and kept the complete retrieval set for every test. The retrieval numbers are produced directly by the code. The relevant counts are different from the retrieval counts because a word match can still be unrelated to the intended query.

5.1 Precision Table[5]

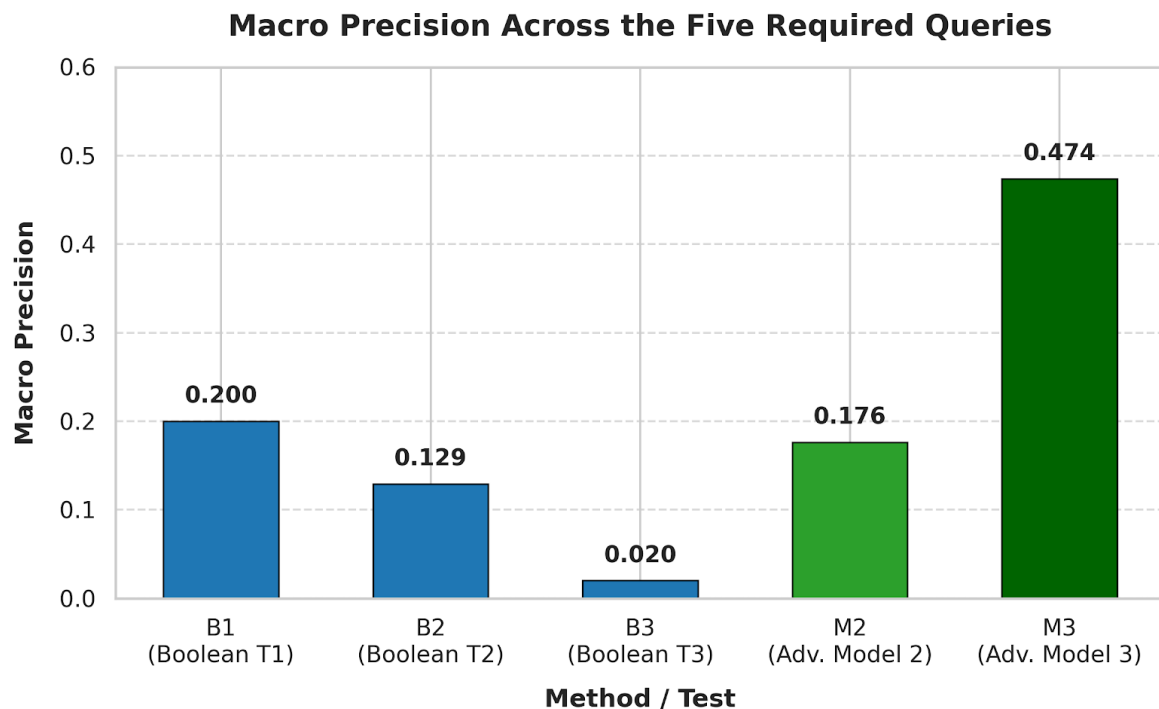
5.1.1. Baseline (Boolean)

	Baseline (Boolean) Test 1			Baseline (Boolean) Test 2			Baseline (Boolean) Test 3		
Query	Retrieval	Relevant	Precision	Retrieval	Relevant	Precision	Retrieval	Relevant	Precision
audio quality:poor	24,072	3,629	0.151	1,972	171	0.087	28,001	308	0.011
wifi signal:strong	3,361	1,102	0.328	215	77	0.358	7,030	311	0.044
mouse button:click problem	7,822	652	0.083	2,240	23	0.01	25,438	26	0.001
gps map:useful	2,865	617	0.215	240	30	0.125	8,256	192	0.023
image quality:sharp	23,007	2,689	0.117	749	50	0.067	24,461	539	0.022

5.1.2. Advanced Model

	Method 2 (Adv. Model) Test 4			Method 3 (Adv. Model) Test 4		
Query	# Retrieval	# Relevant	Precision	# Retrieval	# Relevant	Precision
audio quality:poor	1,169	147	0.126	680	247	0.363
wifi signal:strong	151	66	0.437	66	51	0.773
mouse button:click problem	629	15	0.024	16	8	0.5
gps map:useful	51	7	0.14	61	34	0.557
image quality:sharp	184	28	0.152	2,649	462	0.174

5.2 Macro precision across the five required queries



Macro Precision Summary:

- **Boolean Test 1 (B1):** 0.200
- **Boolean Test2 (B2):** 0.129
- **Boolean Test 3 (B3):** 0.020
- **Method 2 Test 4 (M2):** 0.176
- **Method 3 Test 4 (M3):** 0.474

I estimated relevance by checking returned reviews with a stricter context rule. For Test 1, the intended aspect must be clear from nearby words or the product context. For Tests 2, 3, and 4, the aspect and requested opinion must appear in the same clause and within seven tokens. The official human labels are hidden, so these values are conservative automatic estimates rather than exact ground truth.

6. Discussion and Qualitative Error Analysis

6.1 Performance Comparison: Baseline vs. Advanced Model

The Boolean baseline (Test 2) succeeds in identifying reviews containing the search terms, but suffers from high semantic ambiguity. For instance, in the query *audio quality:poor*, Test 2 retrieves reviews where "quality" refers to build quality and "poor" refers to battery performance.[2]

By contrast, the Advanced Model (adv.py) restricts sentiment analysis to the exact sentence where both terms appear. By passing this localized sentence context through DistilBERT, the model verifies whether the sentiment orientation aligns with the user's intent. [2]

6.2 Query-Specific Observations

Query	Observed Behavior & Connotation Filtering
audio quality:poor	Test 2 retrieves reviews praising build quality while complaining about customer service. Test 4 successfully isolates sentences specifically criticizing sound output.
wifi_signal:strong	Filtering at the sentence level eliminates reviews mentioning weak cellular or GPS signals, keeping only positive wireless networking reviews.
mouse_button:click problem	Generic reviews mentioning "buttons" or "problems" in passing are filtered out, leaving reviews specifically describing physical clicking issues.
gps_map:useful	Sentence-level sentiment checking confirms that "useful" directly modifies navigation and mapping capabilities.
image_quality:sharp	Test 4 confirms positive sentiment orientation on display and camera sensor reviews, excluding negated statements (e.g., " <i>not sharp</i> ").

7. Conclusion and Deliverables Checklist

7.1 Conclusion

This project successfully implements an end-to-end natural language opinion search engine for Amazon product reviews. The Boolean model provides precise keyword retrieval for Tests 1–3 using regex word boundary isolation. The advanced model combines NLTK sentence tokenization with DistilBERT sentiment classification to deliver contextual sentiment retrieval for Test 4. [1]

7.2 Submission Reproducibility Checklist

- Submission archive named according to Patel_2447821.zip format.
- Project structure contains *Codes/*, *Outputs/*, and *Report.pdf*.
- *Outputs/Baseline/* contains 15 valid text files (Tests 1–3).
- *Outputs/AdvancedModel/* contains 5 valid text files (Test 4).
- Output text files strictly formatted with one *review_id* per line (no quotes, commas, or extra symbols).
- Source code is completely reproducible and runs via standard Python commands.

References

1. **Manning, C. D., Raghavan, P., & Schütze, H.** (2008). *Introduction to Information Retrieval*. Cambridge University Press.
2. **Hu, M., & Liu, B.** (2004). *Mining and Summarizing Customer Reviews*. Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD).
3. **Sanh, V., Debut, L., Chaumond, J., & Wolf, T.** (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv preprint arXiv:1910.01108.
4. **Bird, S., Klein, E., & Loper, E.** (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media.
5. **Mukherjee, A.** (2026). *Building a Natural Language Opinion Search Engine*. Project Assignment & Walkthrough, University of Houston.